

Bajaj Game Store

Technical Architecture Documentation

Engineering — Marketing Assist Gamification

May 5, 2026

Contents

1	Executive Summary	3
2	High-Level Architecture	3
2.1	Workspace Layout	3
3	Build Orchestration	4
3.1	Root Scripts	4
3.2	Smart Build (<code>scripts/smart-build.js</code>)	4
3.3	Copy + Manifest (<code>scripts/copy-games.js</code>)	5
4	The Micro-Frontend Architecture (How It Is Authored)	5
4.1	Not Webpack Module Federation	5
4.2	Game Vite Configuration	6
4.3	Shell Mounting Lifecycle	6
5	Routing and Application Lifecycle	6
5.1	Routes (<code>app.routes.ts</code>)	6
5.2	Application Initialisation (<code>app.config.ts</code>)	6
5.3	Per-Environment Manifest URLs	7
6	Authentication	7
6.1	Cipher and Key	7
6.2	Token Payload	7
6.3	End-to-End Flow	8
6.4	Auth Guard	8
6.5	No <code>postMessage</code> Channel	8
7	State Management	8
7.1	Shell — RxJS BehaviorSubject	8
7.2	Games — React Context + <code>useState</code>	9
7.3	Asset Layer	10
8	Inter-Application Communication	10
8.1	Shell to Game (Down-call)	10
8.2	Game to Shell (Up-call)	10

9 Referral Logic	11
9.1 Building the Share URL	11
9.2 Shortening via Vspagy	11
9.3 Sharing UX	12
9.4 Attribution at the LMS	12
9.5 Post-Build Patch (<code>scripts/fix_share_name.js</code>)	12
10 Shared Logic Across Games	12
11 API Consumption	13
11.1 HTTP Clients in Use	13
11.2 LMS Endpoints (Centralised in Each Game's <code>utils/api.js</code>)	13
11.3 LMS Base URLs (per Environment)	13
11.4 Manifest Endpoint	14
11.5 Asset Prefetch	14
12 The Two Dichotomies in the System	14
12.1 Two Game-Mounting Strategies	14
12.2 Two Authentication Modes	14
13 Environments and Deployment	14
13.1 Environment Files	14
13.2 Deployment Targets	15
14 Summary Reference Tables	15

1 Executive Summary

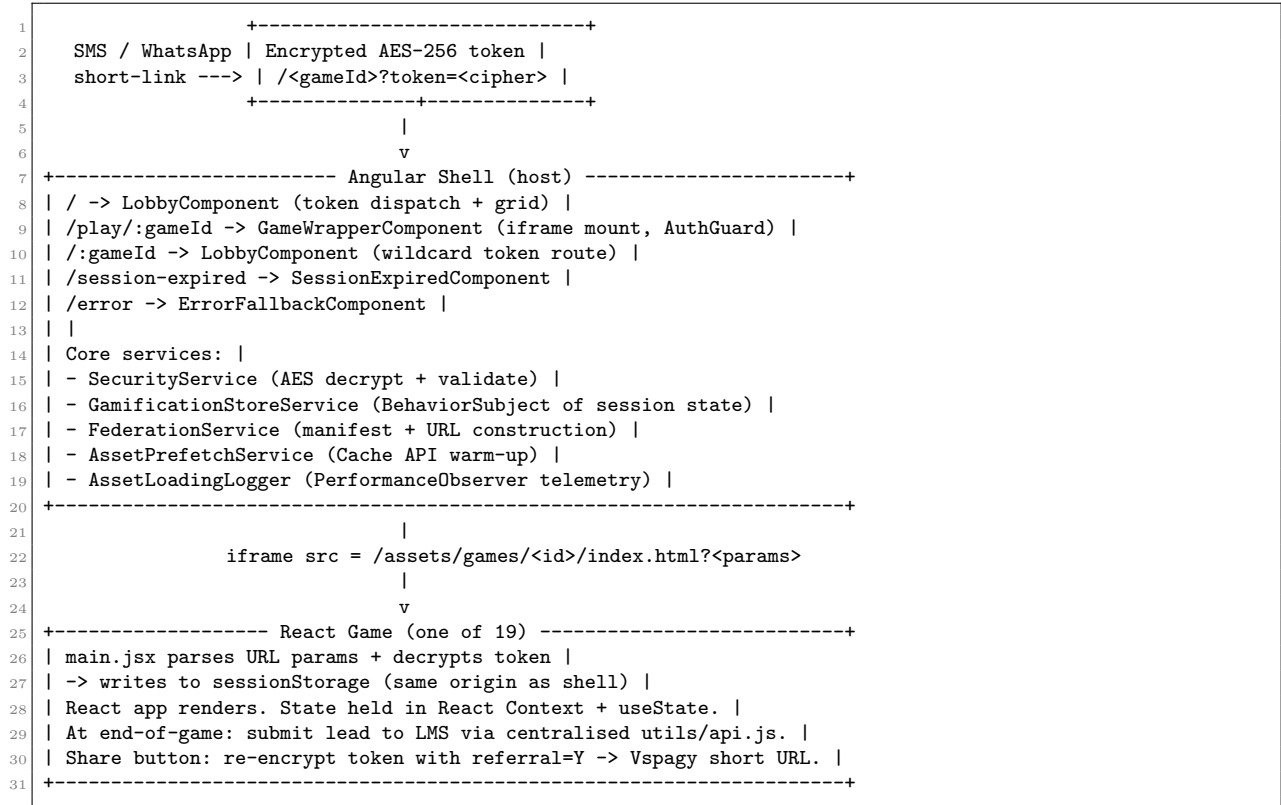
The **Bajaj Game Store** is a monolithic, multi-application gamification platform that ships a single Angular 18 *shell* hosting nineteen independent React 19 + Vite *game* applications. The system is delivered as one deployable artefact built from a **pnpm** workspace; all games are bundled into the shell's **assets/** folder at build time and mounted at runtime in same-origin iframes.

The platform's design goals are:

- **One deploy, one origin.** A single Angular bundle plus all game builds live under one host, eliminating CORS and cross-origin auth concerns.
- **Token-gated personalised access.** A shared AES-256 token delivered via SMS / WhatsApp authenticates the user, identifies the target game, and carries CRM context.
- **Centralised LMS & referral plumbing.** Every game submits leads through a uniform schema (commit 0ce3cd7); referral attribution is encoded into the token itself.
- **Incremental, parallelisable builds.** A smart build script rebuilds only games whose source has changed.

2 High-Level Architecture

Listing 1: Workspace and runtime topology



2.1 Workspace Layout

Listing 2: Root directory layout (pnpm-workspace.yaml)

```
1 bajaj-game-store/
2 |-- angular-shell/ # Angular 18 host (lobby, auth, wrapper)
```

```

3 | -- scripts/
4 | | -- smart-build.js # Incremental, hash-based rebuilder
5 | | -- copy-games.js # Copies dist/ -> shell assets, writes manifest
6 | | -- fix_share_name.js # Post-build patch for share-name displays
7 | -- pnpm-workspace.yaml # 19 games + angular-shell
8 | -- package.json # Root build / dev / serve scripts
9 | -- 19 React + Vite game packages, e.g.:
10 | | -- life-goals/
11 | | -- quiz-game/
12 | | -- Scramble-Words/
13 | | -- life-milestone-race/
14 | | -- retirement-readiness-journey/
15 | | -- retirement-sudoku/
16 | | -- financial-tetris/
17 | | -- Tile-Flipping-game/
18 | | -- life-shield-bomber/
19 | | -- Snake-Life/
20 | | -- secure-saga/
21 | | -- life-snakes-ladders/
22 | | -- life-flight/
23 | | -- one-life/
24 | | -- life-sorted/
25 | | -- tower-defense/
26 | | -- stackability-stack/
27 | | -- health-shield/health-shield/
28 | | -- life-goals-bubble-shooter/

```

3 Build Orchestration

3.1 Root Scripts

Command	Effect
pnpm dev	Starts Angular shell on :4200 and all game dev servers in parallel.
pnpm build	Builds all games (incrementally), copies them into the shell, then builds the shell — single deployable bundle.
pnpm build:uat	Same, with <code>--env uat</code> ; selects each game’s <code>build:uat</code> script and the UAT manifest / LMS URLs.
pnpm build:preprod	Same, with <code>--env preprod</code> .
pnpm build:prod	Same, with <code>--env prod</code> .
pnpm build:force	Forces all games to rebuild regardless of source-hash state.
pnpm copy:games	Re-runs only the copy + manifest writer (no rebuild).

Table 1: Workspace-level build commands (`package.json`).

3.2 Smart Build (`scripts/smart-build.js`)

The smart builder avoids re-building unchanged games by hashing their source.

1. `discoverGames()` reads `pnpm-workspace.yaml`, excludes `angular-shell`.
2. For each game, `computeSourceHash()` walks `src/`, `public/`, root config files, and `.env*` files; concatenates relative paths plus contents and returns an MD5.
3. `getStoredHash()` reads `{gameDir}/dist/.build-hash` from the previous build.
4. If hashes match *and* `dist/` exists $\Rightarrow$ skip. Otherwise spawn `pnpm --filter <game> <buildScript>`, then `saveHash()`.

5. `getBuildScript(env)` maps the env flag (`uat`, `preprod`, `prod`) to the per-game script name (`build:uat`, `build:preprod`, `build:prod`).

3.3 Copy + Manifest (`scripts/copy-games.js`)

After the games are built, `copy-games.js` stitches them into the shell:

1. For every game, recursively copies `{game}/dist/` → `angular-shell/src/assets/games/{gameId}/`. On Windows, `robustRemove()` retries with a trash fallback to survive `ENOTEMPTY/EPERM` file-lock errors.
2. Writes one manifest per environment: `federation.manifest.json`, `federation.manifest.preprod.json`, `federation.manifest.production.json`.
3. Each manifest entry is a static description of where the game lives and what assets to prefetch (see Listing 3).

Listing 3: Federation manifest entry (illustrative)

```
1 {
2   "life-goals": {
3     "remoteEntry": "assets/games/life-goals/index.js",
4     "exposedModule": "../GameEntry",
5     "type": "react",
6     "displayName": "Life Goals Preparedness",
7     "popular": true,
8     "gameId": "GAME_002",
9     "assets": [
10      "assets/games/life-goals/assets/videos/intro.mp4",
11      "assets/games/life-goals/assets/videos/level1.mp4"
12    ]
13  }
14 }
```

4 The Micro-Frontend Architecture (How It Is Authored)

4.1 Not Webpack Module Federation

Despite the manifest's name, the platform does **not** use Webpack Module Federation. There is no runtime `remoteEntry.js` negotiation, no shared dependency graph between shell and games, and no dynamic remote URL.

Instead the platform uses what we will call **static manifest federation**:

- Each game is a fully self-contained Vite build (`base: './`, ES modules, hashed asset filenames).
- Game outputs are physically copied into the shell's `assets/games/` folder.
- The shell discovers them at runtime via the JSON manifest.
- Mounting is performed via an `<iframe>`, not via JS module imports.

This trades runtime flexibility (independent deploys) for operational simplicity (one bundle, one origin, no CORS, no version drift).

4.2 Game Vite Configuration

Each game's `vite.config.js` follows the same template:

Listing 4: Per-game Vite configuration (representative)

```
1 const LMS_URLS = {
2   uat: 'https://sales.bajajlife.com/BalicLmsUtil',
3   preprod: 'https://sales.bajajlife.com/BalicLmsUtil',
4   production: 'https://sales.bajajlife.com/BalicLmsUtil',
5 };
6
7 export default defineConfig(({ mode }) => ({
8   base: './', // relative paths so it mounts under any prefix
9   plugins: [react()],
10  define: {
11    __LMS_BASE_URL__: JSON.stringify(LMS_URLS[mode] || LMS_URLS.uat),
12    __LMS_UPDATE_BASE_URL__: JSON.stringify(LMS_UPDATE_URLS[mode] || LMS_UPDATE_URLS.uat),
13  },
14  build: {
15    outDir: 'dist',
16    rollupOptions: {
17      output: { name: 'LifeGoalsGame', exports: 'named', format: 'es' }
18    }
19  }
20 }));
```

The `define` block bakes environment-specific endpoints into each game at build time, so games never read window configuration at runtime.

4.3 Shell Mounting Lifecycle

`angular-shell/src/app/shared/components/game-wrapper/game-wrapper.component.ts` is responsible for mounting a game.

1. Router activates `GameWrapperComponent` (guarded by `AuthGuard`).
2. Component reads `:gameId` from `ActivatedRoute`.
3. Calls `FederationService.getGameUrl(gameId)` which constructs the `iframe src`.
4. URL is sanitised with `DomSanitizer.bypassSecurityTrustResourceUrl()`.
5. A loading spinner is shown until the `iframe` fires its `load` event; on `error` a `retry / back-to-lobby` panel is rendered.
6. Inside the `iframe`, the game's `main.jsx` parses the query string, decrypts the token if present, primes `sessionStorage`, then bootstraps React.

5 Routing and Application Lifecycle

5.1 Routes (`app.routes.ts`)

5.2 Application Initialisation (`app.config.ts`)

Bootstrap providers, in order:

1. `provideRouter(routes, withPreloading(PreloadAllModules))` — all lazy modules are preloaded after first paint.
2. `APP_INITIALIZER` runs `initializeApp()`:
 - Logs startup banner.

Path	Component	Guarded
/	LobbyComponent	no
/play/:gameId	GameWrapperComponent	yes (AuthGuard)
/:gameId	LobbyComponent (wildcard, handles ?token=)	no
/life-snakes-ladders	GameWrapperComponent	yes (legacy)
/session-expired	SessionExpiredComponent	no
/error	ErrorFallbackComponent	no

Table 2: Angular shell routes.

- `FederationService.loadManifest()` fetches the environment’s manifest URL.
 - Logs game count and popular games.
 - Schedules `AssetPrefetchService.prefetchPopularGames()` 2 s after init (non-blocking, runs in background).
3. `AppComponent.ngOnInit()` starts a 10 s ticker that prints a rolling asset-loading summary via `AssetLoadingLogger`.

5.3 Per-Environment Manifest URLs

Environment	Manifest URL
development	assets/federation.manifest.prod.json
uat	assets/federation.manifest.prod.json
preprod	assets/federation.manifest.preprod.json
production	assets/federation.manifest.production.json

Table 3: Manifest URL chosen per Angular environment file.

6 Authentication

6.1 Cipher and Key

The platform uses **AES-256 in ECB mode** with PKCS7 padding and Base64 encoding. The same key constant is shared between the shell and every game so that the token can be decrypted and re-encrypted on either side.

Listing 5: Shared AES key constant (resides as a literal in source)

```
1 AES_KEY_B64 = 'TKgxQ/0eHM6XRxs1J/PbMy0COu24cH7h4CwpyzQ2T3U='
```

6.2 Token Payload

Listing 6: Decrypted token shape

```
1 {
2   "game_id": "GAME_002",
3   "emp_id": "E12345",
4   "emp_name": "John Doe",
5   "emp_mobile": "9876543210",
6   "location": "Mumbai",
7   "zone": "West",
8   "referral": "N" // "Y" if launched from a share link
9 }
```

6.3 End-to-End Flow

1. User receives a short link of the form `https://<host>/<gameId>?token=<encryptedAES>`.
2. Angular routes to `LobbyComponent` via the wildcard `/:gameId` route.
3. `SecurityService.authenticateWithToken(token)`:
 - Decrypts using `CryptoJS.AES.decrypt` (ECB).
 - Validates required fields.
 - Calls `FederationService.resolveApiGameId(game.id)` to map the API ID (`GAME.002`) to the internal slug (`life-goals`).
 - Constructs `SalesPerson` and `GameDetails` objects.
 - Pushes a `GamificationState` into `GamificationStoreService.setState(...)` along with the raw token.
4. `LobbyComponent.secureNavigateToGame()` sets `window.location.href` to the manifest-derived URL with all relevant query parameters.
5. Inside the iframe, the game's `main.jsx` parses the query string, decrypts the token, populates `sessionStorage`, and rewrites the URL via `history.replaceState()` so the token never appears in the visible address bar.

6.4 Auth Guard

Listing 7: Outline of `AuthGuard.canActivate()`

```
1 canActivate(): boolean {  
2   if (this.store.isValidState()) return true;  
3   this.router.navigate(['/session-expired']);  
4   return false;  
5 }  
6  
7 // isValidState():  
8 // salesPerson.id && gameDetails.id && gameDetails.url
```

6.5 No postMessage Channel

Because shell and game share the same origin, no `postMessage` or `BroadcastChannel` is used. Cross-frame communication is performed implicitly through `sessionStorage` (same origin $\Rightarrow$ shared storage).

7 State Management

7.1 Shell — RxJS BehaviorSubject

`angular-shell/src/app/core/services/gamification-store.service.ts` owns the shell's session state.

Listing 8: `GamificationStoreService` (essence)

```
1 export interface GamificationState {  
2   salesPerson: { id; name; region; mobile; zone };  
3   gameDetails: { id; desc; url; thumbnail };  
4   rawToken: string;  
5   authenticatedAt: number;  
6 }  
7  
8 private stateSubject = new BehaviorSubject<GamificationState | null>(null);  
9  
10 public state$ = this.stateSubject.asObservable();
```

```

11 public salesPerson$ = this.state$.pipe(map(s => s?.salesPerson ?? null));
12 public gameDetails$ = this.state$.pipe(map(s => s?.gameDetails ?? null));
13
14 setState(sp, gd, token): void
15 getSnapshot(): GamificationState | null
16 isValidState(): boolean
17 getConstructedGameUrl(): string
18 clearState(): void

```

getConstructedGameUrl() is dual-mode:

- **Local games** (asset path resolved via manifest): strips trailing /index.html and any query, then appends a query string (?userId=...&gameId=...&token=...).
- **Remote games** (URL baked directly into GameDetails.url): appends path parameters (</userId>/<gameId>).

7.2 Games — React Context + useState

No game uses Redux, Zustand, or Jotai. State is held in a single React Context provider per game; durable values (lead numbers, partial progress) are written to `sessionStorage`.

Listing 9: quiz-game/src/context/QuizContext.jsx — shape

```

1 export const SCREENS = {
2   WELCOME, QUESTION, FEEDBACK, LEAD_CAPTURE, RESULTS, THANK_YOU
3 };
4
5 const QuizProvider = ({ children }) => {
6   // Game state
7   const [currentScreen, setCurrentScreen] = useState(SCREENS.WELCOME);
8   const [questions, setQuestions] = useState(() => getShuffledQuestions());
9   const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0);
10  const [selectedAnswer, setSelectedAnswer] = useState(null);
11  const [userAnswers, setUserAnswers] = useState([]);
12  const [score, setScore] = useState(0);
13
14  // Lead capture
15  const [leadName, setLeadName] = useState('');
16  const [leadPhone, setLeadPhone] = useState('');
17  const [isLeadSubmitted, setIsLeadSubmitted] = useState(false);
18  const [leadNo, setLeadNo] = useState(
19    () => sessionStorage.getItem('quizLeadNo') || null
20  );
21
22  // Re-entry guard for double-clicks
23  const isProcessingRef = useRef(false);
24
25  const onLeadSubmit = useCallback(async (name, phone) => {
26    const result = await submitToLMS({
27      name, mobile_no: phone, score,
28      goal_name: 'Quiz Game',
29      summary_dtls: 'Lead Submission',
30    });
31    if (result.success) {
32      setLeadName(name); setLeadPhone(phone);
33      setIsLeadSubmitted(true);
34      setCurrentScreen(SCREENS.RESULTS);
35    }
36    return result;
37  }, [score]);
38
39  return (
40    <QuizContext.Provider value={{ ...state, ...actions }}>
41      {children}
42    </QuizContext.Provider>
43  );
44 };

```

Other games follow the same pattern with a per-game context, plus small custom hooks under `src/hooks/` (`useSound`, `useLocalStorage`, etc.).

7.3 Asset Layer

- `AssetPrefetchService.prefetchPopularGames()` iterates manifest entries with `popular: true` and fills the `game-assets-v1` `caches.open()` bucket with each entry's `assets[]`.
- `AssetLoadingLogger` patches `document.createElement` and wires a `PerformanceObserver` to maintain a 500-entry circular buffer of resource events; `getFailedAssets()`, `getSummary()`, and `exportLogs()` surface this for debugging.

8 Inter-Application Communication

8.1 Shell to Game (Down-call)

State is handed to a game via two channels:

1. **URL query parameters.** `FederationService.getGameUrl()` constructs an `iframe` `src` of the form

Listing 10: Iframe src constructed by the shell

```
1 /assets/games/life-goals/index.html
2 ?userId=E12345
3 &gameId=life-goals
4 &empName=John+Doe
5 &empMobile=9876543210
6 &location=Mumbai
7 &zone=West
8 &token=<encryptedAES256Token>
```

2. `sessionStorage`, written by the game's `main.jsx` immediately on load (same origin as shell). Keys (used identically by every game):

Key	Source
<code>gamification_userId</code>	query string (<code>userId</code>)
<code>gamification_gameId</code>	query string (<code>gameId</code>)
<code>gamification_empName</code>	query string (<code>empName</code>)
<code>gamification_empMobile</code>	query string (<code>empMobile</code>)
<code>gamification_location</code>	query string (<code>location</code>)
<code>gamification_zone</code>	query string (<code>zone</code>)
<code>gamification_rawToken</code>	query string (<code>token</code> , encrypted)
<code>gamification_game_id</code>	decrypted token field
<code>gamification_emp_id</code>	decrypted token field
<code>gamification_emp_name</code>	decrypted token field
<code>gamification_emp_mobile</code>	decrypted token field
<code>gamification_referral</code>	decrypted token field (Y/N)

Table 4: `sessionStorage` contract — written by every game's `main.jsx` on boot.

8.2 Game to Shell (Up-call)

There is *no synchronous up-call* from a game to the shell. Games never `postMessage` a result or call `window.parent`. Instead:

- Game results are submitted directly to the LMS over HTTPS from inside the iframe.
- Persistent values that should survive a remount (e.g. `quizLeadNo`, `lifeGoalsLeadNo`) are written back to `sessionStorage` for re-reads.
- For navigation (e.g. “return to lobby”), the game just sets `window.top.location.href = '/'`; the shell route changes and the iframe is torn down.

9 Referral Logic

The referral system is built on *token re-encryption* rather than a backend referral table. The same AES key shared between shell and games means any game can mint a new token whose only difference is `referral: 'Y'`.

9.1 Building the Share URL

Implementation lives per game in `src/utils/crypto.js` (the function body is identical across games):

Listing 11: `buildShareUrl()` (e.g. `life-goals/src/utils/crypto.js`)

```

1 export function buildShareUrl() {
2   if (!sessionStorage.getItem('gamification_rawToken')) return null;
3
4   const payload = {
5     game_id: sessionStorage.getItem('gamification_game_id') || '',
6     emp_id: sessionStorage.getItem('gamification_emp_id') || '',
7     emp_name: sessionStorage.getItem('gamification_emp_name') || '',
8     emp_mobile: sessionStorage.getItem('gamification_emp_mobile') || '',
9     location: sessionStorage.getItem('gamification_location') || '',
10    zone: sessionStorage.getItem('gamification_zone') || '',
11    referral: 'Y', // mark as referral
12  };
13
14  const newToken = encryptPayload(payload); // AES-256 ECB, base64
15  const gameId = payload.game_id;
16  const origin = window.location.origin;
17
18  return `${origin}/gamification/${encodeURIComponent(gameId)}${
19    + '?token=${encodeURIComponent(newToken)}';
20 }
```

9.2 Shortening via Vspagy

Each game also ships an identical `src/utils/shortener.js` that proxies the long URL through Vspagy:

```

1 export async function shortenUrl(longUrl) {
2   const response = await fetch('https://vspagy.com/csites/apishortlink/', {
3     method: 'POST',
4     headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
5     body: JSON.stringify({
6       bid: '30144827887',
7       key: '5g2frpb2t2Cxsu/zjt0cRg==',
8       tid: 'LIVE',
9       lid: 'LIVE',
10      lurl: longUrl,
11    }),
12  });
13  if (response.ok) {
14    const data = await response.json();
15    if (data.CODE === '1000') return data.DETAIL; // shortened URL
16  }
17  return longUrl; // fallback
18 }
```

9.3 Sharing UX

The result screen of every game (`ScoreResultsScreen.jsx`, `ResultScreen.jsx`, etc.) calls a near-identical `handleShare()`:

Listing 12: Share button handler (representative)

```
1 const handleShare = async () => {
2   const rawUrl = buildShareUrl() || window.location.href;
3   const shareUrl = await shortenUrl(rawUrl);
4
5   const shareData = {
6     title: 'Life Goals Preparedness',
7     text: 'Hi, I scored ${score}/100 on this game. Try it: ${shareUrl}',
8     url: shareUrl,
9   };
10
11   if (navigator.share) {
12     await navigator.share(shareData); // native sheet on mobile
13   } else {
14     await navigator.clipboard.writeText(shareUrl); // desktop fallback
15   }
16 };
```

9.4 Attribution at the LMS

Once the recipient opens the short link, the chain *Lobby* → *SecurityService* → *GamificationStoreService* is replayed but with `referral === 'Y'`. The downstream lead submission sets `p_source` accordingly:

Listing 13: Branching in `utils/api.js`

```
1 p_source: sessionStorage.getItem('gamification_referral') === 'Y'
2   ? 'Referral'
3   : 'Marketing Assist',
```

This is the entirety of the referral attribution: there is *no* server table, *no* referral code generator, *no* click tracker. The token is the single source of truth.

9.5 Post-Build Patch (`scripts/fix_share_name.js`)

Earlier code paths read the sender’s name from `gamification_emp_name`. The post-build script rewrites all 20 affected result/conversion screens across 10 games to instead read from the captured `leadData?.name`, so the displayed name reflects the real lead, not the salesperson who launched the game. The script runs as part of `pnpm copy:games`.

10 Shared Logic Across Games

There is **no** shared/ or common/ workspace package. Every game ships its own copy of three near-identical utility modules:

File	Responsibility
<code>src/main.jsx</code>	Parses URL, decrypts token, primes <code>sessionStorage</code> , scrubs the URL, then mounts React.
<code>src/utils/crypto.js</code>	<code>decryptToken</code> , <code>encryptPayload</code> , <code>buildShareUrl</code> .
<code>src/utils/shortener.js</code>	Vspagy short-link client.
<code>src/utils/api.js</code>	<code>submitToLMS</code> (new lead) and <code>updateLeadNew</code> (existing lead).

Table 5: The de facto “shared layer” — duplicated per game.

The deliberate design is that any game can be checked out, built, and run independently without depending on a shared library. The trade-off is that a change to the shared protocol (e.g. a new `sessionStorage` key) requires a sweep across all games — this is what `scripts/fix_share_name.js` exists to do safely.

11 API Consumption

11.1 HTTP Clients in Use

- **Shell:** Angular `HttpClient` (used by `FederationService` to fetch the manifest).
- **Games:** the native `fetch` API. No `axios`, no shared interceptor; per-call error handling.

11.2 LMS Endpoints (Centralised in Each Game's `utils/api.js`)

Listing 14: submitToLMS payload

New lead — POST `{__LMS_BASE_URL__}/whatsappInhouse`

```

1 {
2   "cust_name": "<Player Name>",
3   "mobile_no": "<10-digit>",
4   "p_source": "Referral" | "Marketing Assist",
5   "p_data_source": "GAMIFICATION",
6   "p_data_medium": " GAMIFICATION ",
7   "remarks": "Game: <gameID> | Score: <n> | Booking Request",
8   "appointment_date": "DD/MM/YYYY",
9   "appointment_time": "09:00 AM",
10  "userId": "<gamification_userId>",
11  "gameID": "<gamification_gameId>"
12 }
```

Listing 15: updateLeadNew payload

Update existing lead — POST `{__LMS_UPDATE_BASE_URL__}/updateLeadNew`

```

1 {
2   "leadNo": "LN123456",
3   "miscObj1": {
4     "stringval2": "<updated name>",
5     "stringval4": "<booking date>",
6     "stringval5": "<booking time>",
7     "stringval6": "<remarks>",
8     "stringval9": "<mobile>"
9   }
10 }
```

11.3 LMS Base URLs (per Environment)

All games inject the LMS base URL at build time via Vite `define`. The current configuration uses one underlying host across environments; per-environment overrides are wired and ready to diverge:

Env	<code>__LMS_BASE_URL__</code>	<code>__LMS_UPDATE_BASE_URL__</code>
UAT	<code>https://sales.bajajlife.com/BalicLmsUtil</code>	<code>https://sales.bajajlife.com/BalicLmsUtil</code>
Preprod	<code>https://sales.bajajlife.com/BalicLmsUtil</code>	<code>https://sales.bajajlife.com/BalicLmsUtil</code>
Prod	<code>https://sales.bajajlife.com/BalicLmsUtil</code>	<code>https://sales.bajajlife.com/BalicLmsUtil</code>

Table 6: LMS endpoints per environment.

11.4 Manifest Endpoint

The shell's only first-party HTTP call at boot is to its own static manifest asset, e.g. `GET assets/federation.manifest.prod`. There is no auth on this asset; it lists only public game metadata.

11.5 Asset Prefetch

`AssetPrefetchService` performs background GETs against each asset URL listed in popular games' manifest entries and stores them in the `game-assets-v1` Cache bucket. Failures are logged via `AssetLoadingLogger` but never block startup.

12 The Two Dichotomies in the System

The platform is structured around two orthogonal dichotomies that are useful to keep in mind when reasoning about any given request.

12.1 Two Game-Mounting Strategies

1. **Manifest-based, local mount.** `FederationService.getGameUrl(gameId)` looks up the game in the loaded manifest and constructs a same-origin URL into `assets/games/<id>/index.html` with query parameters. This is the path used for *all* bundled games.
2. **Store-constructed remote mount.** When `GameDetails.url` is supplied directly (i.e. a remote third-party game URL), `getConstructedGameUrl()` appends path parameters (`/<userId>/<gameId>`) instead of query parameters. This branch is presently unused by any shipped game but is the extension point for hosting games on external infrastructure.

12.2 Two Authentication Modes

1. **Token-gated mode (sales person / customer).** Triggered when the user lands on `/<gameId>?token=...`. The AES token authenticates the user and carries CRM context. Lead submissions are tagged `p_source: "Marketing Assist"` (or "Referral" if the token's `referral` flag is Y).
2. **Guest mode (lobby-driven).** Triggered when the user clicks "Play" on the lobby grid. The shell calls `GamificationStoreService.setState` with `id: 'GUEST_USER'` and a sentinel `rawToken 'GUEST_SESSION'`. The game receives no real token and skips decryption; lead submissions still flow to the LMS but as anonymous leads.

13 Environments and Deployment

13.1 Environment Files

Listing 16: `angular-shell/src/environments/environment.ts` (production)

```
1 export const environment = {
2   production: true,
3   envName: 'production',
4   apiBaseUrl: 'https://api.marketingassist.com',
5   gameBaseUrl: 'https://bajajlifeinsurance.com/gamification',
6   manifestUrl: 'assets/federation.manifest.production.json',
7 };
```

Listing 17: `environment.uat.ts`

```
1 export const environment = {
2   production: false,
3   envName: 'uat',
```

```

4  apiBaseUrl: 'https://uat-api.marketingassist.com',
5  gameBaseUrl: 'https://balicuat.bajajlifeinsurance.com/gamification',
6  manifestUrl: 'assets/federation.manifest.prod.json',
7  };

```

Listing 18: environment.preprod.ts

```

1  export const environment = {
2    production: false,
3    envName: 'preprod',
4    apiBaseUrl: 'https://bajajuat2-api.marketingassist.com',
5    gameBaseUrl: 'https://balicuat.bajajlifeinsurance.com/gamification',
6    manifestUrl: 'assets/federation.manifest.preprod.json',
7  };

```

13.2 Deployment Targets

- **Docker:** Dockerfile at repo root builds a static image served by nginx.conf.
- **Static hosting / CDN:** upload angular-shell/dist/angular-shell/browser/ as-is.
- **Bare nginx:** drop the same folder under /var/www/html/ and use the bundled nginx.conf.

The full bundle (shell + 19 games + their MP4 assets) is approximately **50–100 MB**, dominated by video.

14 Summary Reference Tables

Concern	Implementation
Build orchestration	scripts/smart-build.js (MD5 source hash, per-game cache); scripts/copy-games.js (recursive copy + manifest writer)
Shell framework	Angular 18 standalone components, RxJS, APP_INITIALIZER
Game framework	React 19 + Vite, ES modules, per-game build (base: './')
Federation strategy	Static manifest federation (no Webpack MF); games mounted in same-origin iframes
Authentication	AES-256 ECB tokens delivered via short link; SecurityService on shell, utils/crypto.js per game
Auth guard	AuthGuard.canActivate → GamificationStoreService.isValidState
Shell state	BehaviorSubject<GamificationState> (gamification-store.service.ts)
Game state	React Context + useState; sessionStorage for durability
Shell-to-game IPC	URL query parameters + same-origin sessionStorage
Game-to-shell IPC	None (no postMessage); games go directly to LMS
Referral logic	Token re-encryption with referral: 'Y' via Vspagy short link; p_source = "Referral" on LMS
Shared logic	Duplicated per game (main.jsx, utils/crypto.js, utils/shortener.js, utils/api.js); no shared workspace package
LMS endpoints	POST /whatsappInhouse (new lead), POST /updateLeadNew (update lead)
Asset prefetch	AssetPrefetchService + Cache API (game-assets-v1) for popular: true games
Telemetry	AssetLoadingLogger + PerformanceObserver circular buffer (500 entries)
Environments	development, uat, preprod, production
Deployment	Single bundle in angular-shell/dist/angular-shell/browser/; Docker, nginx, or any static host

Table 7: Architecture cheat sheet.